

Reviewer Pack — Minimal Order of Motivic Connectivity and Communication Comp...

Entry 5c12ecfe7722 · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-06-04 01:37:18 UTC

Minimal Order of Motivic Connectivity and Communication Complexity Rank

Entry ID: 5c12ecfe7722 **Verdict:** INCONCLUSIVE **Recorded:** 2026-06-04 01:37:18 UTC

1. Conjecture

Field A (mathematical branch): Motivic Theory **Field B** (complexity object): Communication Complexity (Matrix Rank)

Statement:

The minimal motivic connectivity, denoted as $\text{mtr}(C)$, for any circuit C computing a communication complexity function is lower-bounded by the rank of its input matrix, $r(M)$, such that $\text{mtr}(C) \geq \Theta(r(M))$. Moreover, there exists a constructive mapping from the circuit's input matrix to its associated motivic connectivity that can be computed in polynomial time.

Rationale (proposer's reasoning):

Motivic theory provides a framework for studying geometric and arithmetic properties of algebraic varieties. By linking this theory with communication complexity, we may uncover new structural insights into the complexity of computing certain functions. The minimal motivic connectivity captures essential geometric information about the variety associated with the circuit, which could potentially reveal non-trivial connections to the matrix rank of its input.

Taxonomy category: motivic_connectivity_communication_complexity (status at proposal time:)

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): ad175d9569b9cb57

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

The conjecture is supported if, across all circuits C with input matrices M of size up to $n=40$, the correlation between $\text{mtr}(C)$ and $r(M)$ exceeds 0.7, measured using Pearson's correlation coefficient over 30 random seeds.

3. Barrier filter (F1)

Final: PASS

Barrier	Verdict	Confidence	LLM ₁	LLM ₂
RELATIVIZATION	SAFE	0.50	UNCERTAIN	UNCERTAIN
NATURAL_PROOFS	SAFE	0.50	UNCERTAIN	UNCERTAIN
ALGEBRIZATION	SAFE	1.00	SAFE	UNCERTAIN
KARP_LIPTON	SAFE	1.00	UNCERTAIN	SAFE

4. Novelty audit

Verdict: NOVEL against 0 hits across arXiv + Semantic Scholar + ECCC

Search queries (3): - "motivic theory" AND "communication complexity rank" - "minimal motivic connectivity" AND "matrix rank" - "constructive mapping" AND "input matrix" AND "motivic connectivity"

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)

Sandbox: isolated subprocess, stdlib-only, ≤90s wall time per cycle **Execution:** rc=1, elapsed=0.2s

5.1 Generated Python source

```
import random
import math
from itertools import combinations

def run_trial(seed: int) -> dict:
    random.seed(seed)

    def gaussian_elimination(A):
        m, n = len(A), len(A[0])
        rank = 0
        for j in range(n):
            i_max = next((i for i in range(rank, m) if A[i][j] != 0), None)
            if i_max is not None:
                A[rank], A[i_max] = A[i_max], A[rank]
                for i in range(m):
                    if i != rank and A[i][j] != 0:
                        factor = -A[i][j] / A[rank][j]
                        for k in range(n):
                            A[i][k] += factor * A[rank][k]
                rank += 1
        return rank

    def matrix_rank(M):
        return gaussian_elimination(M)

    def communication_complexity_rank(M):
        m, n = len(M), len(M[0])
        max_rank = 0
        for subset in combinations(range(n), m):
            submatrix = [[M[i][j] for j in subset] for i in range(m)]
```

```

        rank = matrix_rank(submatrix)
        if rank > max_rank:
            max_rank = rank
    return max_rank

def generate_random_matrix(n):
    M = [[random.randint(0, 1) for _ in range(n)] for _ in range(n)]
    return M

n_values = [5, 10, 15, 20, 30, 40]
instances_tested = 0
total_mtr_C = 0.0
total_r_M = 0.0
max_n = 0

for n in n_values:
    for _ in range(5):
        M = generate_random_matrix(n)
        mtr_C = communication_complexity_rank(M)
        r_M = matrix_rank(M)

        if mtr_C == 0 or r_M == 0:
            continue

        instances_tested += 1
        total_mtr_C += mtr_C
        total_r_M += r_M
        max_n = n

if instances_tested < 30:
    return {
        "metric_name": "communication_complexity_rank",
        "metric_value": None,
        "instances_tested": instances_tested,
        "n_max": max_n,
        "conjecture_holds": False,
        "counterexample": "insufficient_instances"
    }

mean_mtr_C = total_mtr_C / instances_tested
mean_r_M = total_r_M / instances_tested
covariance = sum((mtr_C - mean_mtr_C) * (r_M - mean_r_M) for mtr_C, r_M in zip(mtr_C_values, r_M_values)) / instances_tested
variance_mtr_C = sum((mtr_C - mean_mtr_C) ** 2 for mtr_C in mtr_C_values) / instances_tested
variance_r_M = sum((r_M - mean_r_M) ** 2 for r_M in r_M_values) / instances_tested

if variance_mtr_C == 0 or variance_r_M == 0:
    return {
        "metric_name": "communication_complexity_rank",
        "metric_value": None,
        "instances_tested": instances_tested,
        "n_max": max_n,
        "conjecture_holds": False,
        "counterexample": "variance_zero"
    }

correlation_coefficient = covariance / (math.sqrt(variance_mtr_C) * math.sqrt(variance_r_M))

```

```

    return {
        "metric_name": "communication_complexity_rank",
        "metric_value": correlation_coefficient,
        "instances_tested": instances_tested,
        "n_max": max_n,
        "conjecture_holds": correlation_coefficient > 0.7,
        "counterexample": ""
    }

if __name__ == "__main__":
    import sys
    seeds = [int(s) for s in sys.argv[1:]] or [2, 3, 5, 7, 11, 13, 17, 19, 23, 29, 31, 37, 41, 43, 47]

    results = []
    for seed in seeds:
        result = run_trial(seed)
        print(f"TRIAL: {result}")
        results.append(result)

    mean_metric_value = sum(r["metric_value"] for r in results if r["metric_value"] is not None) / len(results)
    std_metric_value = math.sqrt(sum((r["metric_value"] - mean_metric_value) ** 2 for r in results) / len(results))
    support_fraction = sum(1 for r in results if r["conjecture_holds"]) / len(results)

    if all(r["conjecture_holds"] for r in results):
        print(f"RESULT: SUPPORTED mean={mean_metric_value} std={std_metric_value} support_fraction={support_fraction}")
    elif any(not r["conjecture_holds"] for r in results):
        first_failing_seed = next(seed for seed, result in zip(seeds, results) if not result["conjecture_holds"])
        print(f"RESULT: FALSIFIED counterexample='{first_failing_seed}' first_failing_seed={first_failing_seed}")
    else:
        print("RESULT: INCONCLUSIVE reason=insufficient_evidence")

```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

```

--STDERR--
Traceback (most recent call last):
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_46663422.py", line 115, in <module>
    result = run_trial(seed)
              ~~~~~^
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_46663422.py", line 84, in run_trial
    covariance = sum((mtr_C - mean_mtr_C) * (r_M - mean_r_M) for mtr_C, r_M in zip(mtr_C_values, r_M_values))
                                                    ~~~~~^
NameError: name 'mtr_C_values' is not defined

```

8. Critic adversarial review

Critic verdict: CHALLENGE

Critic reasoning:

skipped: test crashed before producing data

9. Final verdict & safety rail

Verdict: INCONCLUSIVE

Reasoning:

The test code crashed before producing data, which means that the pre-registered support condition could not be unambiguously met. | next: Review and debug the test code to ensure it runs successfully and produces the required data for further analysis.

11. Audit log (LLM calls)

Total LLM calls: 9

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	propose	ollama_remote	glm4:latest	0	0	14471
2	preregistration	ollama_remote	glm4:latest	0	0	18505
3	novelty	ollama_remote	glm4:latest	0	0	8905
4	novelty	ollama_remote	glm4:latest	0	0	9624
5	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	14862
6	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	17826
7	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	13233
8	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	23301
9	judge	ollama_remote	glm4:latest	0	0	8640

Totals: 0 input tokens, 0 output tokens, ~\$0.0000 cost, 129368 ms total latency. Provider mix: {'ollama_remote': 9}

(full prompt+response transcripts available in research/audit/5c12ecfe7722.jsonl)

12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in research/audit/5c12ecfe7722.jsonl for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: research/pvsnp_sandbox/test_*.py (per-cycle)

Replay tarball: research/replay/5c12ecfe7722.tar.gz (if generated)